

RB-mrico-java-001：Mongo导致系统繁忙故障处理 Runbook

Metadata

- 环境: 预发布uat
- 服务名称: general-device-public
- 告警名称: 系统繁忙
- 负责人: @sre-oncall
- 最后更新: 2026-07-08
- 下次演练: xxx
- 预计耗时: 5-20 分钟
- 严重级别: P1 (全量不可用)

触发条件

- ☑ 用户反馈: Web/App 页面显示"系统繁忙，请稍后重试"

前置检查清单

- ☑ 已连接uat的jumpserver
- ☑ kubectl 上下文已切换至 prod 集群或者登陆对应环境的 kuboard
- ☑ 已打开监控大盘: <http://192.168.13.131:3000/>
- ☑ 已通知值班群: 「<service-name> 系统繁忙故障排查中，@oncall」

快速诊断

步骤 1：确认故障范围

```
# 查看服务 Pod 状态
kubectl get pods -n general -l app=general-device-public

# 查看服务整体可用性
kubectl get svc general-device-public -n general
```

NAME	READY	STATUS	RESTARTS	AGE
general-device-public-57f8dc4b4f-n1dcw	1/1	Running	0	2d

判断：

- STATUS 非 Running → 步骤 3：Pod 级故障
- READY 非 1/1 → 步骤 3：Pod 级故障
- 全部正常 → 步骤 2

步骤 2: 查看 JVM 指标

```
# 进入 Pod (任选一个)
kubectl exec -it general-device-public-57f8dc4b4f-nlbcw -n general -- /bin/sh

# 查看 JVM 堆内存
jcmd 1 VM.heap_summary

# 或查看 GC 情况
jstat -gcutil 1 1000 5
```

步骤 3: JVM 内存 / GC 问题

3.1 快速确认

```
# 在 Pod 内执行
jmap -heap 1

# 查看堆中对象统计 (找内存大户)
jmap -histo 1 | head -30
```

3.2 处理

A: 堆内存不足 (Heap Usage > 85%)

```
# 紧急措施: 水平扩容, 分摊压力
kubectl scale deployment <service-name> --replicas=8 -n prod

# 或垂直扩容 (临时提升内存 limit)
kubectl patch deployment <service-name> -n prod -p '{
  "spec": {
    "template": {
      "spec": {
        "containers": [{
          "name": "<service-name>",
          "resources": {
            "limits": {"memory": "2Gi"}
          }
        }]
      }
    }
  }
}'
```

B: Full GC 频繁

```
# 查看 GC 日志（如已配置 -Xloggc）
cat /app/logs/gc.log | tail -50

# 生成 Heap Dump（谨慎，会 STW）
jmap -dump:format=b,file=/tmp/heap.hprof 1

# 将 dump 文件拷贝到本地分析
kubectl cp prod/<pod-name>:/tmp/heap.hprof ./heap.hprof
```

注意：生成 Heap Dump 会触发 STW，建议在业务低峰期或已扩容后执行。

C：内存泄漏

- `jstat -gcutil` 中 `0` (Old Gen) 是否持续增长且 GC 后不回降
- 分析 Heap Dump

步骤 4：线程 / 连接池问题

4.1 线程状态检查

```
# 查看线程 Dump
jstack 1 > /tmp/thread-dump.txt

# 统计线程状态
grep java.lang.Thread.State /tmp/thread-dump.txt | sort | uniq -c
```

关键判断：

线程状态	正常占比	异常表现	处理
<code>RUNNABLE</code>	大部分	-	正常
<code>BLOCKED</code>	< 5%	大量线程阻塞	检查锁竞争 / 死锁
<code>WAITING</code> / <code>TIMED_WAITING</code>	部分	大量线程等待	检查连接池 / 下游超时
<code>Deadlock</code>	0	发现死锁	重启服务 + 代码修复

4.2 连接池耗尽检查

```
# 查看线程 Dump 中是否有大量线程卡在连接池获取
grep -A 5 "getConnection\|druid\|HikariPool" /tmp/thread-dump.txt | head -50
```

常见原因：

- 数据库连接池配置过小 (`maxActive` / `maximumPoolSize`)
- 慢 SQL 导致连接长时间占用
- 下游 HTTP 调用超时设置不合理，线程被长时间占用

应急处理：

```
# 临时扩大连接池（需配合配置中心或环境变量）
# 或重启 Pod 释放连接（最后手段）
kubectl rollout restart deployment/<service-name> -n prod
```

故障发现

1 下游依赖故障

```
# 查看 Pod 事件
kubectl describe pod general-device-public-57f8dc4b4f-nlbcw -n general | tail
-40

# 查看 Pod 日志（最近 200 行 ERROR）
kubectl logs general-device-public-57f8dc4b4f-nlbcw -n general --tail=200 | grep
-E "ERROR|FATAL|Exception"

Caused by: com.mongodb.MongoQueryException: Query failed with error code 96 with
name 'OperationFailed' and error message 'Executor error during find command ::
caused by :: Sort operation used more than the maximum 33554432 bytes of RAM. Add
an index, or specify a smaller limit.' on server 192.168.13.77:27017

# 或查看上一个容器日志（崩溃重启时）
kubectl logs general-device-public-57f8dc4b4f-nlbcw -n general --previous --
tail=100
```

下游服务	故障表现	处理
MongoDB副本集	more than the maximum 33554432 bytes of RAM.	链接：RB-OPS-003: MongoDB内存限制故障处理 Runbook

```
# 查看服务调用链路（如有 Skywalking / Jaeger）
# 或查看日志中的 Feign / RestTemplate 调用超时

kubectl logs <pod-name> -n prod --tail=500 | grep -E "timeout|connection
refused|Read timed out|ServiceUnavailable"
```

验证

- `kubectl get pods -n prod -l app=<service-name>` → 全部 Running & Ready
- 用户侧"系统繁忙"提示消失

回滚方案

```
# 如应急措施后问题加剧，执行回滚
kubectl rollout undo deployment/<service-name> -n prod

# 确认回滚状态
kubectl rollout status deployment/<service-name> -n prod
kubectl get pods -n prod -l app=<service-name>
```

